

GITOPS · KUBERNETES · OBSERVABILIDADE

Plataforma GitOps Kubernetes Multi-Ambiente — Dev + Produção

Especificação, arquitetura e guia de implementação de um laboratório GitOps completo:
Kubernetes (kind/cloud-agnostic) · ArgoCD · Prometheus & Grafana · Headlamp · cert-
manager · CoreDNS · GitHub Actions — desenhado com rigor de produção desde o
primeiro commit (segurança, observabilidade e promoção dev → prod via Pull Request).

Kubernetes (kind / cloud-agnostic)

ArgoCD · App of Apps

Prometheus + Grafana

Headlamp

cert-manager · TLS

CoreDNS / external-dns

GitHub Actions CI/CD

Kustomize base + overlays

Sealed Secrets

Velero backup

Repositório: [GitOps-LAB-Kubernetes-Observabilidade](#) · Documento gerado para planejamento
e execução faseada

SUMÁRIO

- 01 **PROMPT — Especificação / Blueprint** O que construir, requisitos por componente, critérios de aceite
- 02 **ARQUITETURA — Visão Geral e Decisões** Diagramas, comparação dev↔prod, justificativas técnicas
- 03 **GUIA DE IMPLEMENTAÇÃO — Passo a Passo** 12 fases com critério de saída objetivo, comandos e troubleshooting

DOCUMENTO 1 DE 3 · PROMPT.MD

PROMPT — Especificação / Blueprint da Plataforma

PROMPT: Plataforma GitOps Kubernetes Multi-Ambiente — Dev + Produção

OBJETIVO GERAL

Você é um engenheiro DevOps/Platform Engineer sênior. Sua tarefa é construir, dentro deste repositório (`GitOps-LAB-Kubernetes-Observabilidade`), uma **plataforma GitOps de referência** que funcione tanto em um **laboratório local (dev)** quanto possa ser promovida, com mudanças mínimas e auditáveis, para um **cluster de produção cloud-agnostic**.

Diferente de um "script gigante que faz tudo de uma vez", o resultado deve ser um repositório GitOps real: declarativo, versionado, idempotente, com separação clara entre **base** (o que é igual em qualquer lugar) e **overlay** (o que muda por ambiente), seguindo o padrão **Kustomize** `base/` + `overlays/<env>/` orquestrado pelo ArgoCD via **App of Apps / ApplicationSets**.

Stack:

- Cluster Kubernetes (dev: `kind` 3 nós; prod: qualquer cluster K8s $\geq$ 1.28 — EKS/GKE/AKS/k3s/bare-metal)
- Ingress Controller (NGINX)
- cert-manager (TLS — self-signed em dev, Let's Encrypt em prod)
- ArgoCD (GitOps CD, App of Apps + ApplicationSets por ambiente)
- GitHub Actions (CI: build, test, scan, assinatura de imagem, promoção de versão)
- kube-prometheus-stack — Prometheus + Grafana + Alertmanager (observabilidade)
- Headlamp (dashboard web do cluster)
- CoreDNS customizado (dev) / external-dns + DNS real (prod)
- Sealed Secrets ou External Secrets Operator (gestão de segredos sem segredo em texto puro no Git)
- Velero (backup e disaster recovery)
- NetworkPolicies (deny-all por padrão + exceções explícitas)
- Pod Security Standards (`baseline` em dev, `restricted` em prod)

PRINCÍPIOS QUE GUIAM ESTA VERSÃO (o que muda em relação a um "bootstrap script só pra dev")

1. **Um único conjunto de manifests, dois (ou mais) ambientes.** Nada de duplicar YAML — tudo que é específico de ambiente (domínio, réplicas, recursos, classe de armazenamento, emissor de certificado, política de sync) vive em `overlays/<env>/`, nunca espalhado pela base.

2. **GitOps de verdade.** O estado desejado do cluster é 100% derivado do conteúdo deste repositório. `kubectl apply` manual é proibido após o bootstrap inicial — toda mudança passa por PR + ArgoCD sync.
3. **Cloud-agnostic em produção.** Nenhum manifesto de prod deve depender de uma API proprietária de nuvem (ALB Ingress Controller da AWS, Cloud DNS do GCP etc.). Use abstrações padrão (`Ingress`, `LoadBalancer` /MetalLB, `cert-manager` com `HTTP01` / `DNS01` configurável, `external-dns` com provider plugável). Documente os pontos de extensão para quem quiser amarrar a um provider específico depois.
4. **Segurança desde o dia 1, não como item de TODO.** `restricted` Pod Security Standard, NetworkPolicy default-deny, RBAC de menor privilégio, segredos nunca em texto puro no Git, imagens escaneadas e assinadas, no `cluster-admin` para apps.
5. **Idempotência e reprodutibilidade.** Bootstrap pode rodar N vezes sem quebrar nada. Nenhuma etapa manual além de "rodar o script" e "abrir o PR".
6. **Observabilidade como produto, não acessório.** Dashboards e alertas versionados como código (`ConfigMap` / `PrometheusRule` / `GrafanaDashboard`), não clicados manualmente na UI.

ESTRUTURA DE DIRETÓRIOS ESPERADA

```

.
├─ clusters/
│   └─ dev/
│       ├── kind-config.yaml      # cluster kind (1 control-plane + 2 workers)
│       └─ README.md             # como subir o cluster local
│   └─ prod/
│       └─ README.md             # requisitos do cluster alvo (versão, addons, storageclass...)
├─ infra/
│   ├── base/                   # manifests/Kustomizations cloud-agnostic e env-agnostic
│   │   ├── namespaces/
│   │   ├── ingress-nginx/
│   │   ├── cert-manager/
│   │   ├── argocd/
│   │   ├── monitoring/         # kube-prometheus-stack (HelmRelease/values base)
│   │   ├── headlamp/
│   │   ├── coredns/           # patch base do CoreDNS
│   │   ├── network-policies/
│   │   ├── secrets-management/ # Sealed Secrets / External Secrets Operator
│   │   └─ backup/             # Velero
│   └─ overlays/
│       ├── dev/               # *.local.dev, TLS self-signed, 1 réplica, retenção curta
│       │   ├── ingress-nginx/
│       │   ├── cert-manager/
│       │   ├── argocd/
│       │   ├── monitoring/
│       │   └─ kustomization.yaml
│       └─ prod/               # domínio real, Let's Encrypt, HA, retenção longa, PSS restricted
│           ├── ingress-nginx/
│           ├── cert-manager/
│           ├── argocd/
│           ├── monitoring/
│           └─ kustomization.yaml
├─ argocd/
│   └─ apps/
│       ├── app-of-apps-dev.yaml # raiz do ambiente dev
│       ├── app-of-apps-prod.yaml # raiz do ambiente prod
│       └─ applicationset-platform.yaml # ApplicationSet que gera Apps por ambiente/cluster
├─ observability/
│   ├── dashboards/            # JSON dos dashboards Grafana (as code)
│   ├── alerts/                # PrometheusRule (PodCrashLooping, HighMemory, etc.)
│   └─ slo/                    # definições de SLO/SLI (opcional, fase 2)

```

```

├─ dns/
│   ├── coredns-custom-cm.yaml      # zona *.local.dev (dev)
│   └─ hosts-snippet.txt           # bloco sugerido para /etc/hosts (dev)
│
├─ github-actions/.github/workflows/
│   ├── ci.yaml                    # build, test, trivy scan, cosign sign, push GHCR
│   └─ cd-promote.yaml             # promote tag de dev → prod via PR automatizado
│
├─ sample-app/
│   ├── src/
│   ├── Dockerfile
│   └─ k8s/
│       ├── base/                  # Deployment, Service, Ingress, HPA, NetworkPolicy
│       └─ overlays/{dev,prod}/
│
├─ scripts/
│   ├── check-prerequisites.sh      # valida docker/kubecthl/helm/kind/jq etc.
│   ├── bootstrap-dev.sh           # sobe kind + aplica app-of-apps-dev (idempotente)
│   ├── teardown-dev.sh            # destrói o cluster dev com segurança
│   └─ smoke-test.sh               # valida endpoints, pods Ready, certificados emitidos
│
├─ docs/
│   ├── ARQUITETURA.md
│   ├── GUIA-DE-IMPLEMENTACAO.md
│   ├── RUNBOOK.md                 # procedimentos operacionais (rotacionar secret, restaurar backup)
│   └─ diagrams/                   # diagramas Mermaid versionados
│
├─ Makefile                        # make up | down | status | logs | port-forward | smoke-test
└─ README.md

```

REQUISITOS TÉCNICOS DETALHADOS

1. Clusters

Dev — `kind`:

- 1 control-plane + 2 workers, `extraPortMappings` para 80/443, CoreDNS habilitado.
- Subido via `scripts/bootstrap-dev.sh`, que precisa ser **idempotente**: se o cluster já existe, reaproveita; se não, cria.

Prod — cloud-agnostic:

- Não provisionamos o cluster em si neste repositório (isso é trabalho de IaC/Terraform, fora de escopo aqui) — documentamos em `clusters/prod/README.md` os **requisitos mínimos**: versão do K8s, StorageClass com `ReadWriteOnce` dinâmico, um mecanismo de `LoadBalancer` (cloud LB nativo ou MetalLB on-prem), e acesso de saída para `letsencrypt-prod` e GHCR.
- O overlay `overlays/prod/` assume apenas essas garantias — nada provider-specific.

2. Namespaces, RBAC e Pod Security Standards

- Namespaces: `ingress-nginx`, `argocd`, `monitoring`, `dashboard`, `cert-manager`, `apps`, `external-secrets` (ou `kube-system` para Sealed Secrets), `velero`.
- Labels de **Pod Security Standards** por overlay:
 - dev → `pod-security.kubernetes.io/enforce: baseline`
 - prod → `pod-security.kubernetes.io/enforce: restricted`
- RBAC de menor privilégio: `ServiceAccount dev-reader` em `apps` com `ClusterRole view`; nenhuma aplicação recebe `cluster-admin`.

3. Ingress Controller (NGINX)

- Helm values base + overlay. Base define métricas habilitadas (porta 10254), headers de segurança (`X-Frame-Options`, `Content-Security-Policy`, etc.) e `resources.requests/limits`.
- Overlay dev: `hostNetwork: true` / `NodePort` (compatível com `kind`).
- Overlay prod: `service.type: LoadBalancer`, `replicaCount ≥ 2` com `PodDisruptionBudget` e `topologySpreadConstraints`.

4. cert-manager + TLS

- Base instala CRDs e o controller.
- Overlay dev: `ClusterIssuer selfsigned-issuer` + `Certificate` wildcard `*.local.dev`.
- Overlay prod: `ClusterIssuer letsencrypt-prod` configurável para `HTTP01` (padrão, cloud-agnostic) ou `DNS01` (plugável por provider, documentado mas não hard-coded).

5. ArgoCD (GitOps Engine)

- Base: Helm values comuns (RBAC, `configs.repositories`, Notifications controller, ResourceCustomizations para health checks customizados).
- Overlay dev: `server.insecure: true` (TLS termina no Ingress local), 1 réplica, Redis standalone.
- Overlay prod: TLS ponta a ponta, `server.replicas ≥ 2`, **Redis HA**, `applicationsetController` habilitado, SSO via OIDC.

- **App of Apps + ApplicationSet:** um `ApplicationSet` gera as `Application` filhas a partir de um *generator* de diretórios (`overlays/{dev,prod}/...`), eliminando duplicação manual de manifestos de Application por ambiente.
- `syncPolicy.automated.{prune,selfHeal}` habilitado em dev; em prod, `selfHeal` habilitado mas com `syncWindows` e `Notifications` para alertar antes/depois de cada sync em horário comercial.

6. CI/CD com GitHub Actions

`ci.yaml` — Build, Test, Scan, Sign, Push:

- `actions/checkout`, build multi-stage com Buildx + cache GHA.
- Testes automatizados do `sample-app` antes do build da imagem.
- **Scan de vulnerabilidades com Trivy** (falha o pipeline em CRITICAL/HIGH não whitelisted).
- **Assinatura de imagem com cosign** (keyless, via OIDC do GitHub) — produção só roda imagens assinadas (verificado por admission policy, ex.: Kyverno/Sigstore policy-controller — fase 2).
- Push para GHCR com tags `latest` + `sha` + `semver` (quando aplicável).

`cd-promote.yaml` — Promoção dev → prod:

- Em vez de `sed` direto no manifesto (frágil e sem revisão), abre **Pull Request automatizado** no próprio repo GitOps atualizando `sample-app/k8s/overlays/prod/kustomization.yaml` (`images: newTag: <sha>`), exigindo aprovação humana antes do merge.
- ArgoCD detecta o merge em `main` e sincroniza prod (`selfHeal` + `prune`).

7. Observabilidade (kube-prometheus-stack)

- Base: Prometheus + Grafana + Alertmanager com `ServiceMonitor` s para ArgoCD e NGINX Ingress.
- Overlay dev: retenção 7d, armazenamento `emptyDir` /StorageClass local pequena, 1 réplica, `adminPassword` lido de Secret local (nunca hard-coded em texto puro, nem em dev).
- Overlay prod: retenção 30d+, `volumeClaimTemplate` com StorageClass dinâmica de 20Gi+, Alertmanager com 3 réplicas e roteamento real (Slack/email/PagerDuty via Secret externo).
- **Dashboards e alertas como código:** JSON em `observability/dashboards/`, `PrometheusRule` em `observability/alerts/` (`PodCrashLooping`, `HighMemoryUsage`, `CertificateExpiringSoon`, `ArgoCDSyncFailed`, `IngressHighErrorRate`, etc.).

8. Dashboard Web do Cluster (Headlamp)

- Overlay dev: acesso via `Ingress` com TLS self-signed, autenticação via token de ServiceAccount.
- Overlay prod: autenticação via **OIDC** (mesmo provider do ArgoCD, se possível), RBAC read-only por padrão.

9. DNS

- Dev: ConfigMap de patch do CoreDNS interno (zona `*.local.dev` → service do Ingress) + snippet de `/etc/hosts` para a máquina host. `dnsmasq` documentado como opcional para wildcard automático.
- Prod: `external-dns` cloud-agnostic (suporta múltiplos providers via configuração), criando registros reais a partir das anotações dos `Ingress` / `Service`. Nenhum hard-code de provider — escolha feita via `values` do overlay.

10. Gestão de Segredos

- **Nenhum segredo em texto puro no Git, em nenhum ambiente.** Adote **Sealed Secrets** (mais simples para o lab, criptografa no client e descriptografa só no cluster) **ou External Secrets Operator** (busca de um vault externo — mais alinhado a prod real). Documentar a escolha e o trade-off; implementar ao menos a primeira opção de ponta a ponta.

11. Network Policies

- `default-deny-all` por namespace de aplicação, com exceções explícitas (`allow-ingress-from-nginx`, `allow-dns-egress`, `allow-monitoring-scrape`).
- Habilitado em ambos os ambientes — dev precisa replicar o comportamento de prod para que bugs de política sejam descobertos cedo.

12. Backup & Disaster Recovery (Velero)

- Instalado via base; overlay prod aponta para um *object storage* compatível com S3 (qualquer provider — MinIO on-prem incluso) e agenda backups diários dos namespaces críticos (`argocd`, `monitoring`, `apps`).
- Documentar e **testar** o procedimento de restore no `docs/RUNBOOK.md`.

BOOTSTRAP E VALIDAÇÃO

`scripts/bootstrap-dev.sh` deve, em ordem e de forma idempotente:

1. Validar pré-requisitos (`scripts/check-prerequisites.sh`: docker, kubectl, helm, kind, jq, cosign...).
2. Criar (ou reaproveitar) o cluster `kind`.
3. Instalar o ArgoCD mínimo (bootstrap "chicken-and-egg").
4. Aplicar o `app-of-apps-dev.yaml` — **a partir daqui, o ArgoCD assume o controle** de tudo o mais (namespaces, ingress, cert-manager, monitoring, headlamp, sample-app...).
5. Aguardar e validar saúde das `Application`s (`argocd app wait` ou polling via `kubectl`).
6. Rodar `scripts/smoke-test.sh`: valida endpoints HTTPS, certificados emitidos, pods `Ready`, dashboards do Grafana acessíveis, ArgoCD sincronizado.
7. Imprimir credenciais iniciais (ArgoCD admin, Grafana) e URLs de acesso.

Makefile expõe: `make up`, `make down`, `make status`, `make logs`, `make port-forward`, `make smoke-test`.

SEGURANÇA E BOAS PRÁTICAS (não-negociáveis)

1. Pod Security Standards: `baseline` (dev) / `restricted` (prod) — `runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `seccompProfile: RuntimeDefault`.
2. NetworkPolicy deny-all + exceções explícitas, em ambos os ambientes.
3. `resources.requests` e `resources.limits` obrigatórios em todo `Deployment` / `StatefulSet`.
4. Nenhum segredo em texto puro no Git — Sealed Secrets ou External Secrets Operator.
5. Imagens escaneadas (Trivy) e assinadas (cosign) no CI; admission policy valida assinatura antes de agendar pods em prod (documentar como evolução de fase 2 caso não dê tempo de implementar).
6. RBAC mínimo — nunca `cluster-admin` para aplicações.
7. Backups testados (Velero) com runbook de restore validado manualmente ao menos uma vez.

PRÉ-REQUISITOS DA MÁQUINA HOST

```
docker          # >= 24.x (rodar em WSL2/Linux — kind exige daemon Docker)
kubectl         # >= 1.28
helm            # >= 3.12
kind            # >= 0.20
kustomize       # (ou kubectl kustomize embutido)
cosign          # assinatura de imagens
trivy           # scan de vulnerabilidades local (opcional, CI já roda)
git, curl, jq
```

Validar tudo via `scripts/check-prerequisites.sh` antes do bootstrap.

ENTREGÁVEIS / CRITÉRIOS DE ACEITE

- ☐ Repositório segue o padrão `base/` + `overlays/{dev,prod}` sem duplicação de manifesto
- ☐ `make up` sobe o ambiente dev local do zero, de forma idempotente, em < ~15 minutos
- ☐ Todos os serviços acessíveis via HTTPS em `*.local.dev` (dev) com certificado válido (mesmo que self-signed) e sem editar `/etc/hosts` manualmente além do bootstrap inicial

- ☐ ArgoCD sincronizando automaticamente a partir do App of Apps (`selfHeal` + `prune`)
- ☐ Pipeline CI completo: build → test → scan (Trivy) → assinatura (cosign) → push GHCR
- ☐ Promoção dev → prod via Pull Request automatizado, não via `sed` direto no manifesto
- ☐ Grafana com dashboards versionados (cluster, pods, NGINX, ArgoCD, certificados)
- ☐ Alertas (`PrometheusRule`) versionados cobrindo cenários críticos comuns
- ☐ NetworkPolicy default-deny ativa, com exceções documentadas, em ambos os ambientes
- ☐ Pod Security Standards aplicados (`baseline` dev / `restricted` prod)
- ☐ Segredos geridos via Sealed Secrets/External Secrets — zero segredo em texto puro no Git
- ☐ Backup configurado (Velero) com procedimento de restore documentado e testado
- ☐ `smoke-test.sh` validando o ambiente ponta a ponta, usável em CI também
- ☐ `docs/ARQUITETURA.md` , `docs/GUIA-DE-IMPLEMENTACAO.md` e `docs/RUNBOOK.md` completos



OBSERVAÇÕES FINAIS

- Gere diagramas de arquitetura em **Mermaid**, versionados em `docs/diagrams/`.
- Documente explicitamente **o que muda** ao promover de dev para prod — isso é o coração deste design (overlay diff deve ser pequeno e legível).
- Cada decisão "cloud-agnostic" deve vir acompanhada de uma nota de "como adaptar para AWS/GCP/Azure especificamente", para não deixar o operador real na mão.
- Releases e mudanças de infraestrutura passam por PR — trate este repositório com o mesmo rigor de revisão que código de aplicação.

DOCUMENTO 2 DE 3 · ARQUITETURA.MD

ARQUITETURA — Visão Geral, Diagramas e Decisões

Arquitetura – Plataforma GitOps Kubernetes (Dev + Produção)

Este documento explica **o que** estamos construindo, **por que** as decisões foram tomadas dessa forma, e **como** as peças se encaixam. Para o passo a passo de implementação, veja `GUIA-DE-IMPLEMENTACAO.md`. Para a especificação que orienta a construção, veja `../PROMPT.md`.

1. Visão Geral

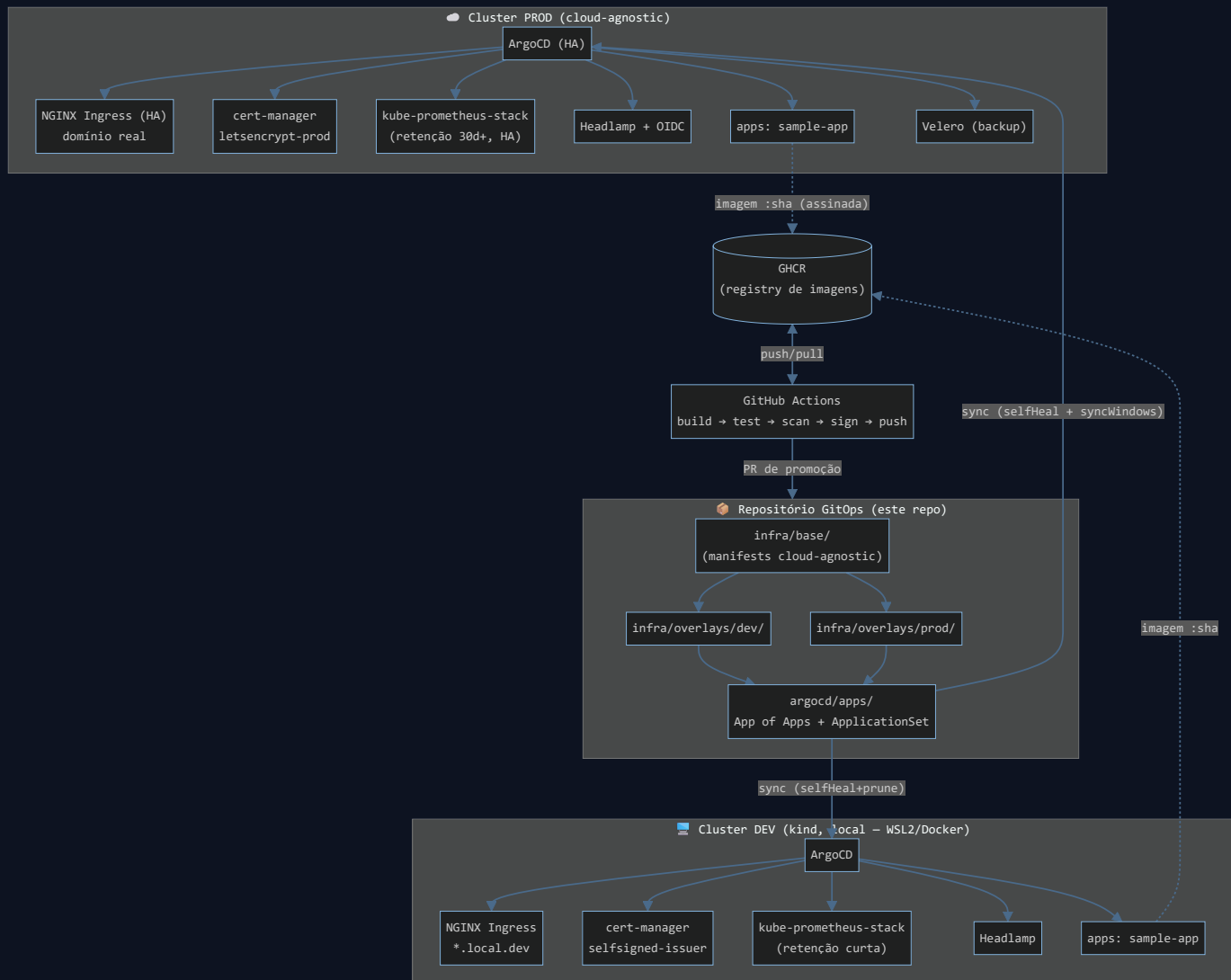
Estamos construindo um **laboratório GitOps que espelha um ambiente de produção real**, não um "demo descartável". A ideia central:

Um único repositório descreve o estado desejado de **N ambientes**. A diferença entre "rodar no meu notebook" e "rodar em produção" deve ser um **diff pequeno e legível** entre `overlays/dev/` e `overlays/prod/` — nunca um conjunto de manifestos paralelos e divergentes.

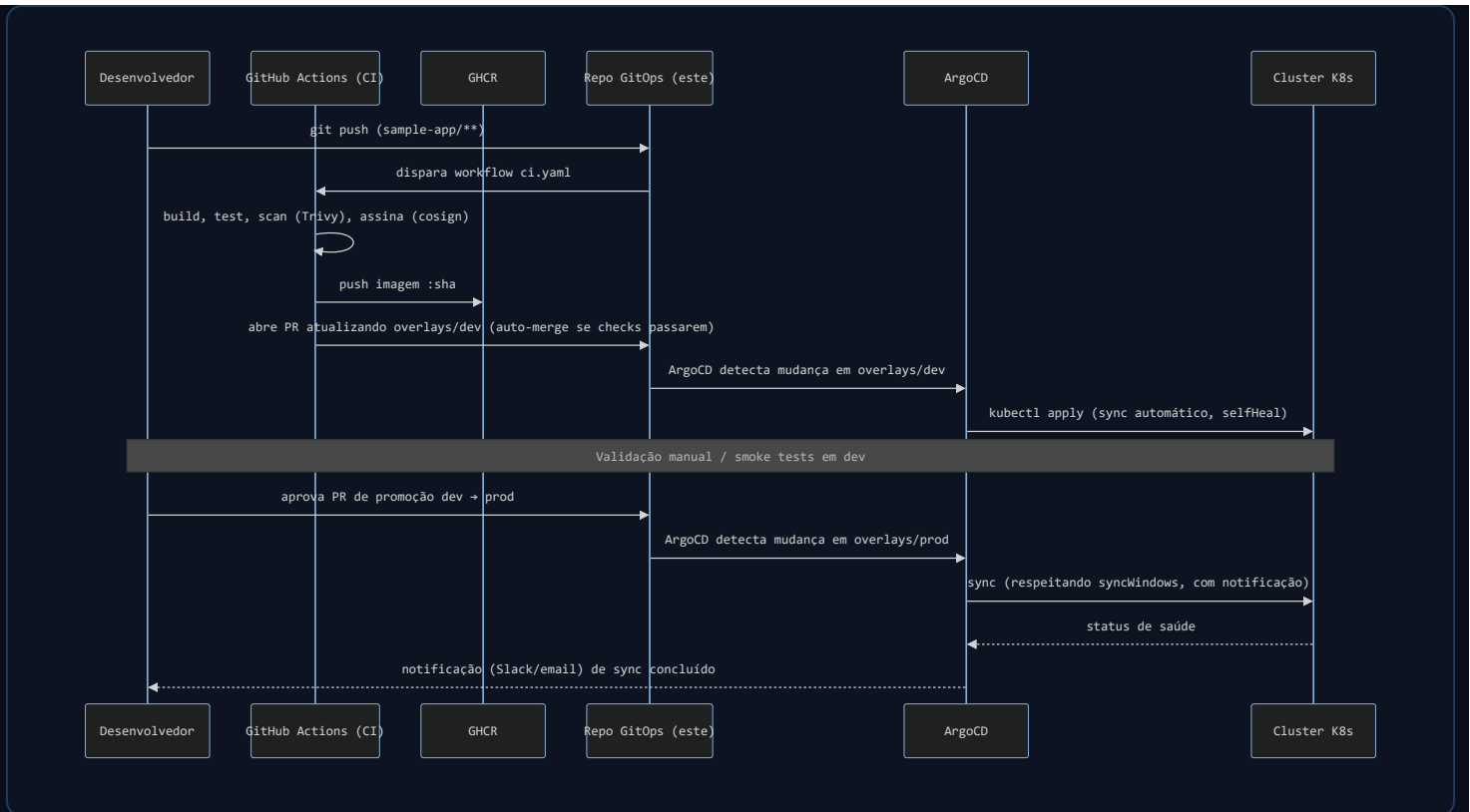
Isso é alcançado combinando três padrões consolidados da comunidade Kubernetes:

Padrão	Papel aqui
Kustomize <code>base/</code> + <code>overlays/</code>	Elimina duplicação: a base descreve "o quê", o overlay descreve "como, neste ambiente"
GitOps com ArgoCD (App of Apps / ApplicationSet)	O cluster nunca é alterado por <code>kubectl apply</code> manual — apenas reconcilia o que está no Git
Pipeline CI/CD com promoção via PR	Imagens são construídas uma vez, escaneadas, assinadas, e promovidas entre ambientes por revisão humana, não por script silencioso

2. Diagrama de Arquitetura (visão de componentes)



3. Fluxo de mudança (do código ao cluster)



A diferença crucial em relação a um pipeline ingênuo: **nada aplica diretamente no cluster**. O CI só constrói, testa, escaneia e abre PRs. Quem aplica é o ArgoCD, reconciliando o que está em `main`. Isso dá auditoria completa (todo `kubectl apply` tem um commit e um PR por trás).

4. Por que Kustomize `base/` + `overlays/` (e não Helm puro, nem repos separados) ?

Avaliamos três abordagens (ver também a pergunta que orientou esta decisão):

1. **Kustomize base + overlays (escolhida)** — a base contém o que é comum (recursos, labels, estrutura), e cada overlay aplica *patches* (`patchesStrategicMerge`, `images`, `replicas`, `configMapGenerator`) só com o que muda. Funciona nativamente com ArgoCD (`kustomize build` é suportado out-of-the-box) e com Helm charts via `helmCharts:` no `kustomization.yaml`, então não perdemos a riqueza dos charts da comunidade (kube-prometheus-stack, ingress-nginx, argo-cd).
2. Helm `values-dev.yaml` / `values-prod.yaml` — mais simples à primeira vista, mas tende a duplicar blocos inteiros de YAML entre arquivos de values, e não generaliza bem para manifests que não vêm de chart (ex.: `Application` do ArgoCD, `NetworkPolicy`, `PrometheusRule`).
3. Repos/branches separados por ambiente — isola bem, mas qualquer correção de bug precisa ser replicada manualmente em N lugares; é o oposto de "single source of truth".

A combinação **Kustomize por cima + Helm chart como fonte (via `helmCharts:`) por baixo** nos dá o melhor dos dois mundos: usamos os charts oficiais e bem mantidos da comunidade, e customizamos por ambiente de forma declarativa e sem duplicação.

5. Por que cloud-agnostic em produção?

Este laboratório roda hoje em **WSL2 num Windows Server**, mas o objetivo de "produção" é servir como modelo adaptável a **qualquer cluster real** — gerenciado (EKS/GKE/AKS) ou on-prem/bare-metal (k3s, kubeadm). Hard-codar para um provider específico (ex.: AWS ALB Ingress Controller + Route53 + ACM) tornaria o repositório inútil fora daquele contexto.

Em vez disso:

- **Ingress**: usamos a API padrão `Ingress` + NGINX Ingress Controller — funciona em qualquer cluster. Quem estiver na AWS pode trocar por ALB depois sem reescrever a aplicação.
- **LoadBalancer**: em clusters gerenciados, o `Service type: LoadBalancer` provisiona o LB da nuvem automaticamente; on-prem, documentamos **MetalLB** como peça plugável.
- **DNS**: `external-dns` suporta dezenas de providers via configuração — não escrevemos código específico de nenhum.
- **TLS**: `cert-manager` com `ClusterIssuer` Let's Encrypt — `HTTP01` funciona em qualquer lugar com Ingress público; `DNS01` é documentado como opção plugável por provider de DNS.

Cada decisão "agnostic" no `PROMPT.md` vem acompanhada de uma nota de "como adaptar para um provider específico" — então quem for de fato colocar isso na AWS, por exemplo, sabe exatamente os 3-4 pontos de extensão a trocar.

6. Diferenças DEV vs PROD (o "diff" que importa)

Esta tabela é a especificação dos overlays — cada linha vira um patch Kustomize.

Aspecto	<code>overlays/dev/</code>	<code>overlays/prod/</code>
Domínio	<code>*.local.dev</code> (DNS local / <code>/etc/hosts</code>)	domínio real via <code>external-dns</code>
TLS	<code>cert-manager</code> self-signed	<code>cert-manager</code> + Let's Encrypt (<code>letsencrypt-prod</code>)
Réplicas (Ingress, ArgoCD, Grafana...)	1	≥ 2 + <code>PodDisruptionBudget</code>
Armazenamento (Prometheus, ArgoCD Redis)	StorageClass local / <code>emptyDir</code> , retenção curta (7d)	StorageClass dinâmica, retenção 30d+, Redis HA
Pod Security Standard	<code>baseline</code>	<code>restricted</code>
<code>ArgoCD.server.insecure</code>	<code>true</code> (TLS termina no Ingress local)	<code>false</code> (TLS ponta a ponta)
Autenticação (Headlamp/Grafana/ArgoCD)	local / token de ServiceAccount	OIDC / SSO corporativo
<code>syncPolicy</code> ArgoCD	<code>automated</code> , <code>{prune, selfHeal}</code> sem restrição	<code>automated.selfHeal</code> + <code>syncWindows</code> (evita sync em horário de pico) + Notifications
Alertmanager	desabilitado ou rota local	roteamento real (Slack/email/PagerDuty)
Backup (Velero)	opcional/desabilitado	obrigatório, com agendamento e teste de restore
Recursos (<code>requests</code> / <code>limits</code>)	mínimos (laptop-friendly)	dimensionados para carga real, com HPA onde fizer sentido
Network egress para registry/Let's Encrypt	livre (lab)	revisado/whitelisted conforme política da organização

7. Decisões de segurança e o porquê

Decisão	Motivação
<code>restricted</code> Pod Security Standard em prod	Bloqueia contêineres rodando como root, com filesystem gravável, ou com escalonamento de privilégio — a maior fonte de "container escape" trivial
NetworkPolicy <code>default-deny</code> + exceções explícitas, em ambos os ambientes	Se só existisse em prod, bugs de política (ex.: app esquece de liberar egress para o DB) só seriam descobertos em produção. Replicar em dev os torna parte do ciclo normal de desenvolvimento
Sealed Secrets / External Secrets Operator	Segredo em texto puro no Git é uma das causas mais comuns de vazamento — uma vez commitado, está no histórico para sempre, mesmo que removido depois
Scan (Trivy) + assinatura (cosign) de imagens no CI	Garante que só imagens sem vulnerabilidades críticas conhecidas, e <i>de fato construídas pelo nosso pipeline</i> (não adulteradas), cheguem ao cluster
RBAC mínimo / nunca <code>cluster-admin</code> para apps	Limita o "raio de explosão" de uma aplicação comprometida
Promoção via PR (não <code>sed</code> direto)	Garante revisão humana e trilha de auditoria para toda mudança que vai para produção

8. Observabilidade como produto

Em vez de instalar o `kube-prometheus-stack` e parar por aí, tratamos dashboards e alertas como **artefatos versionados**:

- `observability/dashboards/*.json` — exportados do Grafana e versionados; carregados via `dashboardProviders` (sidecar do chart), nunca criados manualmente na UI.
- `observability/alerts/*.yaml` — `PrometheusRule`s cobrindo cenários operacionais reais: `PodCrashLooping`, `HighMemoryUsage`, `CertificateExpiringSoon`, `ArgoCDSyncFailed`, `IngressHighErrorRate`, `NodeDiskPressure`.
- `ServiceMonitor`s para ArgoCD e NGINX Ingress garantem que a própria plataforma também é observável — não só as aplicações.

Isso faz da observabilidade parte do "contrato" do repositório: qualquer novo componente adicionado precisa vir com seu `ServiceMonitor` / `PrometheusRule` / dashboard, do mesmo jeito que viria com seu manifesto de `Deployment`.

9. Limitações conhecidas e próximos passos (fase 2)

Documentamos honestamente o que fica de fora do escopo inicial:

- **Provisionamento do cluster de produção em si** (Terraform/Cluster API) — fora de escopo; documentamos os requisitos mínimos em `clusters/prod/README.md`.
- **Admission policy validando assinatura cosign** (Kyverno/Sigstore policy-controller) — desenhado, mas implementado como follow-up caso o tempo não permita na primeira iteração.
- **SLOs/SLIs formais** — pasta `observability/slo/` reservada, conteúdo é evolução futura.
- **Multi-cluster / multi-region** — o `ApplicationSet` já está desenhado para crescer nessa direção (generator de cluster), mas a topologia inicial é single-cluster por ambiente.

10. Onde cada pergunta encontra resposta

Pergunta	Onde procurar
"O que exatamente vamos construir e em que ordem?"	<code>GUIA-DE-IMPLEMENTACAO.md</code>
"Qual a especificação/contrato de cada componente?"	<code>../PROMPT.md</code>
"Como eu, operador, faço uma rotina do dia a dia (rotacionar secret, restaurar backup)?"	<code>RUNBOOK.md</code> <i>(criado durante a implementação)</i>
"Por que decidimos X em vez de Y?"	Este documento (seções 4, 5, 6, 7)

DOCUMENTO 3 DE 3 · GUIA-DE-IMPLEMENTACAO.MD

GUIA DE IMPLEMENTAÇÃO — Passo a Passo Faseado

Guia de Implementação – Plataforma GitOps Kubernetes (Dev + Produção)

Passo a passo prático: o que será feito, em que ordem, com quais comandos, e como validar cada etapa antes de seguir para a próxima. Este guia assume o repositório descrito em `../PROMPT.md` e as decisões explicadas em

`ARQUITETURA.md`.

0. Diagnóstico do ambiente atual (já levantado)

Antes de qualquer coisa, este é o estado real do host onde vamos rodar o ambiente `dev`:

Item	Situação encontrada	Implicação
OS	Windows Server 2025 + WSL2 (Ubuntu-24.04, running)	O cluster <code>kind</code> precisa rodar dentro do WSL2 (kernel Linux + Docker), não no Windows
Docker	✅ já instalado e funcionando dentro do <code>Ubuntu-24.04</code> (v29.5.3, usuário <code>server</code> no grupo <code>docker</code>)	Não precisa instalar — (atualiza a nota "pendente: instalar Docker" do setup, que está desatualizada)
kubectl / helm / kind / kustomize / cosign / trivy	❌ nenhum instalado	Precisamos instalar — passo 1 abaixo
Portas 80 / 443 / 6443	✅ livres	<code>kind</code> pode mapear sem conflito
RAM / Disco	98 GB RAM (93 GB livres) / ~933 GB livres em <code>/</code>	Sobra de sobra para um cluster <code>kind</code> de 3 nós + toda a stack
<code>sudo</code>	pede senha (não é passwordless)	Os scripts de instalação que exigem <code>sudo</code> vão pedir senha interativamente — não dá para automatizar 100% sem o usuário digitar a senha uma vez
Containers já rodando no WSL2	Ollama, Open WebUI, Postgres, Portainer, Prometheus (9090) , Grafana (3000) , node-exporter, cadvisor, nginx (8080)	⚠️ atenção: o <code>kube-prometheus-stack</code> vai instalar <i>seu próprio</i> Prometheus/Grafana dentro do cluster — eles não competem por porta com os containers Docker existentes (acesso será via Ingress/ <code>*.local.dev</code> , não via <code>localhost:3000</code> / <code>:9090</code>), mas é bom ter isso em mente ao depurar "por que a Grafana que abri não é a do cluster"

💡 Nota: a memória do projeto registra "Pendente: Instalar Docker Engine no Ubuntu 24.04 (WSL2)" — isso **já foi resolvido** (Docker 29.5.3 rodando). Vamos atualizar essa anotação.

1. Fase 0 — Ferramentas de linha de comando (dentro do WSL2

`Ubuntu-24.04`)

Tudo a partir daqui roda **dentro do WSL2** (`wsl -d Ubuntu-24.04`), porque é onde o Docker e o kernel Linux estão.

Instalar (versões mínimas indicadas no `PROMPT.md`):

```
# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind

# kustomize (opcional — kubectl já embute "kubectl kustomize")
curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh"

# cosign + trivy
go install github.com/sigstore/cosign/v2/cmd/cosign@latest # ou binário pré-compilado do GitHub Releases
sudo apt-get install -y trivy # ou script oficial aquatone/trivy
```

Critério de saída desta fase: `scripts/check-prerequisites.sh` (criado na Fase 1) roda sem erros e imprime as versões de todas as ferramentas.

2. Fase 1 — Esqueleto do repositório

Criar a estrutura de diretórios completa descrita no `PROMPT.md` (vazia/com placeholders), e os primeiros scripts utilitários:

1. `scripts/check-prerequisites.sh` — valida `docker`, `kubectl`, `helm`, `kind`, `jq`, `cosign`, `trivy`; falha com mensagem clara apontando o que falta.
2. `clusters/dev/kind-config.yaml` — cluster de 3 nós (1 control-plane + 2 workers), `extraPortMappings` 80→80 e 443→443.
3. `Makefile` com os targets `up`, `down`, `status`, `logs`, `port-forward`, `smoke-test` (inicialmente "esqueleto", preenchidos conforme as fases avançam).

Critério de saída: `make status` roda (mesmo que ainda reporte "cluster não existe") sem erro de sintaxe; estrutura de diretórios espelha o `PROMPT.md`.

3. Fase 2 — Subir o cluster `kind` (dev)


```
kind create cluster --config clusters/dev/kind-config.yaml --name devcluster
kubectl cluster-info --context kind-devcluster
kubectl get nodes -o wide          # esperar 3 nós em Ready
```

Critério de saída: `kubectl get nodes` mostra 3 nós `Ready`; `kind get clusters` lista `devcluster`.

Idempotência: o `bootstrap-dev.sh` deve checar `kind get clusters | grep devcluster` antes de tentar criar — se já existe, pula esta etapa.

4. Fase 3 — Bootstrap do ArgoCD ("ovo e galinha")

ArgoCD precisa existir no cluster *antes* de poder gerenciar o resto via GitOps — por isso ele é instalado uma vez, manualmente/via script, e a partir daí **assume o controle de tudo**.

```
kubectl create namespace argocd
helm upgrade --install argocd argo-cd \
  --repo https://argoproj.github.io/argo-helm \
  --namespace argocd \
  -f infra/base/argocd/values.yaml \
  -f infra/overlays/dev/argocd/values-patch.yaml

kubectl -n argocd wait --for=condition=available --timeout=300s deployment/argocd-server
```

Critério de saída: `kubectl -n argocd get pods` mostra todos os pods `Running`; `argocd-server` acessível via `port-forward` antes mesmo do Ingress existir:

```
kubectl -n argocd port-forward svc/argocd-server 8081:443
# abrir https://localhost:8081 — login admin / senha do secret argocd-initial-admin-secret
```

5. Fase 4 — Aplicar o "App of Apps" (a partir daqui, é tudo GitOps)

```
kubectl apply -f argocd/apps/app-of-apps-dev.yaml
```

A partir deste ponto, **nenhum outro** `kubectl apply` / `helm install` **deve ser feito manualmente** — o ArgoCD reconcilia tudo a partir do Git: namespaces, RBAC, NetworkPolicies, NGINX Ingress, cert-manager, kube-prometheus-stack, Headlamp, CoreDNS patch, Sealed Secrets, Velero, sample-app.

Critério de saída: `argocd app list` (ou `kubectl get applications -n argocd`) mostra todas as `Application`s com `SYNC STATUS = Synced` e `HEALTH STATUS = Healthy`. Isso pode levar alguns minutos na primeira sincronização (pull de imagens, CRDs do cert-manager etc.).

Se alguma `Application` ficar `Degraded` / `OutOfSync` por muito tempo, **não** corrija com `kubectl edit` direto no cluster — corrija o manifesto no Git e deixe o ArgoCD ressincronizar. Isso é o próprio propósito do `selfHeal`: o cluster sempre volta a refletir o Git.

6. Fase 5 — DNS local e validação de acesso HTTPS

1. Aplicar o snippet de `/etc/hosts` (dentro do WSL2 e/ou no Windows, conforme onde o navegador roda) apontando `*.local.dev` para `127.0.0.1`.
2. Confirmar que o patch do CoreDNS foi aplicado (`kubectl -n kube-system get cm coredns -o yaml`).
3. Acessar cada serviço via HTTPS e confirmar que o certificado (self-signed) foi emitido:

```
curl -kv https://argocd.local.dev      # esperar handshake TLS OK + redirect/login
curl -kv https://grafana.local.dev
kubectl get certificate -A              # READY=True em todos
```

Critério de saída: todos os hosts em `*.local.dev` respondem via HTTPS com certificado emitido pelo `selfsigned-issuer`, sem erro de conexão recusada.

7. Fase 6 — Observabilidade: validar dashboards e alertas como código

1. Confirmar que os `ConfigMap`s/ `PrometheusRule`s de `observability/` foram sincronizados pelo ArgoCD (não criados manualmente na UI).
2. Acessar `https://grafana.local.dev`, confirmar que os dashboards de `observability/dashboards/*.json` aparecem na pasta "Kubernetes" sem import manual.
3. Validar que os alertas existem no Prometheus/Alertmanager:

```
kubectl -n monitoring get prometheusrules
# Abrir https://prometheus.local.dev/alerts e conferir PodCrashLooping, HighMemoryUsage etc.
```

Critério de saída: dashboards e alertas aparecem **sem nenhuma ação manual na UI** — tudo veio do Git via ArgoCD. Esse é o teste decisivo de "observabilidade como código".

8. Fase 7 — Sample app e fluxo de CI completo

1. Implementar o `sample-app` (API simples) com `Dockerfile`, testes, manifests `base` / `overlays`.
2. Configurar os *secrets* do GitHub Actions (`GITHUB_TOKEN` já existe; adicionar `COSIGN_PRIVATE_KEY` /OIDC keyless, e o token de PR automatizado).
3. Disparar o pipeline (`git push` em `sample-app/**`):
 - build → test → scan Trivy → assina com cosign → push GHCR → abre PR atualizando `sample-app/k8s/overlays/dev/kustomization.yaml` (`images.newTag`).
4. Mergear o PR → ArgoCD detecta e sincroniza → novo pod sobe em `https://app.local.dev`.

Critério de saída: alterar uma linha no `sample-app`, dar `git push`, e ver — sem nenhum passo manual no cluster — a nova versão rodando em `app.local.dev` em poucos minutos, com o histórico completo (commit → PR → sync) auditável no GitHub e no ArgoCD.

9. Fase 8 — Hardening de segurança

1. Aplicar `NetworkPolicy` default-deny + exceções; validar com testes negativos (`kubectl run` um pod de teste e confirmar que o tráfego não permitido é bloqueado).
2. Confirmar labels de Pod Security Standard (`baseline` em dev) e que pods que violam o padrão são rejeitados (testar deliberadamente com um manifesto "ruim").
3. Migrar qualquer segredo em texto puro para Sealed Secrets/External Secrets — `git grep` por padrões de segredo deve retornar zero resultados fora de `*.sealed.yaml` /referências ao operador externo.
4. Rodar `trivy` localmente contra as imagens usadas e revisar achados.

Critério de saída: `git grep -i -E "(password|secret|token|api[_-]?key)\s*[:=]\s*['\"](?:[^\"]|\"")"` não encontra segredo em texto puro em nenhum manifesto versionado.

10. Fase 9 — Backup/Restore (Velero) e Runbook

1. Configurar Velero apontando para um *object storage* compatível com S3 (MinIO local serve para o teste em dev).
2. **Executar de fato** um backup e um restore de teste (ex.: apagar o namespace `apps` e restaurá-lo a partir do backup) — documentar o procedimento exato em `docs/RUNBOOK.md`.

Critério de saída: restore testado manualmente ao menos uma vez, com passos exatos documentados — "documentado mas nunca testado" não conta como pronto.

11. Fase 10 — `smoke-test.sh` e `make up` ponta a ponta

Consolidar tudo em `scripts/smoke-test.sh`, que valida (e pode rodar em CI também):

- todos os nós `Ready`;
- todas as `Application`s do ArgoCD `Synced` / `Healthy`;
- todos os `Certificate`s `Ready=True`;
- endpoints HTTPS respondendo (`argocd`, `grafana`, `prometheus`, `dashboard`, `app`);
- dashboards/alertas presentes no Grafana/Prometheus.

Depois, validar que **o processo inteiro é idempotente e reproduzível do zero**:

```
make down      # destrói o cluster dev
make up        # recria do zero — deve funcionar sem intervenção manual
make smoke-test # valida tudo automaticamente
```

Critério de saída: `make down && make up && make smoke-test` roda do zero, sem intervenção manual além de digitar a senha do `sudo` quando solicitado, e termina com "tudo verde".

12. Fase 11 — Overlay de produção e fluxo de promoção

Esta fase **não exige um cluster de produção real** para ser validada — o objetivo é que o overlay `prod` seja **renderizável e coerente**:

```
kubectl kustomize infra/overlays/prod | kubectl apply --dry-run=client -f -
```

1. Revisar o *diff* `kubectl kustomize overlays/dev` vs `overlays/prod` — deve ser pequeno, legível, e bater exatamente com a tabela da seção 6 de `ARQUITETURA.md`.

2. Escrever `clusters/prod/README.md` com os requisitos mínimos do cluster alvo.
3. Implementar e testar o workflow `cd-promote.yaml` (abre PR de promoção dev → prod) — pode ser testado contra o próprio cluster dev simulando "prod" num segundo namespace, se não houver cluster de produção disponível.

Critério de saída: `kubectl kustomize infra/overlays/prod` renderiza sem erro, o diff para `dev` é exatamente o esperado, e o fluxo de PR de promoção foi testado ao menos uma vez de ponta a ponta (mesmo que o destino final seja simulado).

13. Ordem recomendada de execução (resumo)

```
Fase 0 → instalar ferramentas (kubectl, helm, kind, kustomize, cosign, trivy)
Fase 1 → criar esqueleto de diretórios + scripts utilitários
Fase 2 → subir cluster kind
Fase 3 → bootstrap manual do ArgoCD (chicken-and-egg)
Fase 4 → aplicar App of Apps – ArgoCD assume o controle
Fase 5 → validar DNS local + HTTPS/TLS
Fase 6 → validar observabilidade como código (dashboards/alertas)
Fase 7 → sample-app + pipeline CI completo (build→test→scan→sign→push→PR→sync)
Fase 8 → hardening (NetworkPolicy, PSS, secrets geridos)
Fase 9 → backup/restore (Velero) testado e documentado
Fase 10 → smoke-test + make down/up idempotente
Fase 11 → overlay de produção renderizável + fluxo de promoção testado
```

Cada fase tem um **critério de saída objetivo** — não avance para a próxima sem cumpri-lo. Isso evita o cenário clássico de "infra que só funciona na primeira vez que alguém rodou".

14. Quando algo der errado — primeiros passos de troubleshooting

Sintoma	Onde olhar primeiro
<code>kind create cluster</code> falha	<code>docker info</code> dentro do WSL2 — daemon rodando? Conflito de porta 80/443/6443?
<code>Application</code> em <code>OutOfSync</code> / <code>Degraded</code>	<code>argocd app diff <nome></code> e <code>kubectl describe application <nome> -n <nome></code> <code>argocd</code> — nunca corrigir com <code>kubectl edit</code> direto
Certificado não fica <code>Ready</code>	<code>kubectl describe certificate <nome></code> e <code>kubectl logs -n cert-manager deploy/cert-manager</code> — geralmente é DNS/HTTP01 challenge não alcançável
<code>*.local.dev</code> não resolve	conferir <code>/etc/hosts</code> (WSL2 e Windows, dependendo de onde o navegador roda) e o patch do CoreDNS
Pod não sobe por Pod Security / NetworkPolicy	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code> — geralmente a mensagem de violação é explícita
CI falha no scan Trivy	revisar achados — não silenciar com <code>--exit-code 0</code> ; corrigir a base image ou adicionar exceção justificada e versionada

15. Próximos passos após este guia

Quando o usuário disser "**pode começar**", a implementação seguirá exatamente esta ordem, fase por fase, com checkpoint de validação ao final de cada uma antes de prosseguir — nada de gerar 40 arquivos de uma vez sem testar incrementalmente.